

Requirements Engineering
Assignment 5

Requirements Traceability

ParcelResolve
Lost or Delayed Parcel Resolution System

Group: Varun & Thanh

Varun Ganesh Sondur
Pham Thanh

October 2026

Contents

Introduction	3
A Meaningful Relationships Between Artefacts	3
A.1 Identified Trace Relationships	3
A.1.1 Stakeholder Needs → Requirements	3
A.1.2 Product Goals / Vision → Requirements	3
A.1.3 Functional ↔ Non-Functional Requirements	4
A.1.4 Requirements → Assignment 4 Review Findings	4
A.1.5 Original → Revised Requirements	4
A.1.6 Requirements → Lifecycle Stages / Scenarios	4
A.1.7 Functional Requirement Dependencies	5
B Traceability Representation	5
B.1 Traceability Matrix	5
B.2 Traceability Information Model	6
B.3 Traceability Chain	6
C Why the Selected Trace Links Are Useful	8
C.1 Requirements Understanding	8
C.2 Change Management and Impact Analysis	8
C.3 Requirements Review and Validation	9
C.4 Maintenance and Evolution	9
C.5 Scope Control	9
D Tool / Approach Supporting Requirements Traceability	10
D.1 Existing Requirements Management Tools	10
D.2 LLM-Supported Traceability Approach	10
D.3 Benefits and Challenges	11
E Maintaining Trace Links as Requirements Evolve — Literature Review	11
E.1 Traceability and Maintenance / Change Management	12
E.2 Automated / ML / LLM-Based Traceability	12
E.3 Continuous or Evolving Traceability	12
E.4 Implications for ParcelResolve	13
F Observations and Conclusions	13

G Review Decision Record	13
References	14
AI Assistance Disclosure	15

Introduction

In Assignments 2–4 we produced the vision, scope and stakeholder analysis (A2), 28 functional and 16 non-functional requirements (A3), and a model-based validation of the incident-resolution lifecycle (A4). Each assignment refers to the previous one, but the references so far are implicit. This assignment makes them explicit and investigates how they can be kept correct when requirements change.

The report follows the assignment tasks. Section A identifies the relationships between our artefacts, Section B represents them as a traceability matrix and a traceability information model, Section C explains why the links are useful, Section D investigates a tool-supported approach, and Section E reviews literature on maintaining links as requirements evolve. Section F concludes, and Section G keeps the AI-supported review separate from our own analysis.

Evidence tags. Because we must not claim links that our earlier work does not support, every link in the matrix carries a tag: [A2], [A3] or [A4] means the relationship is documented in that assignment; [CAND.] means it is a candidate link proposed in this assignment and not yet confirmed against the source text.

A Meaningful Relationships Between Artefacts

A.1 Identified Trace Relationships

The artefacts from the earlier assignments are: the stakeholders ST-001 to ST-005 and the product vision with its lifecycle (A2); FR-01 to FR-28 and NFR-01 to NFR-16 (A3); and the behavioural model, validation questions VQ-01 to VQ-14, scenarios S-01 to S-12 and review decisions C-01 to C-10 (A4). The following subsections describe the relationship types between them. Representative individual links are listed in Table 3.

A.1.1 Stakeholder Needs → Requirements

A2 identified five stakeholders: ST-001 Host / Customer Organization, ST-002 Product Owner, ST-003 External Delivery Stakeholders, ST-004 Operational User and ST-005 IT / Integration Team. A requirement without a stakeholder behind it is hard to justify, and a stakeholder need without a requirement may be forgotten. The link answers the question “who asked for this?”.

Our A3 requirements cover incident initiation, classification, information and status, workflow, action and responsibility, evidence, monitoring, escalation, customer communication, resolution and closure, and operational insights. The link from ST-005 to the integration-failure requirement NFR-15 is the clearest example: the IT / Integration Team is the stakeholder most affected by how the system behaves when an integration fails. We list this as a candidate link (M-01) until the exact A2 need statement is checked.[\[confirm\]](#)

A.1.2 Product Goals / Vision → Requirements

The main product value in A2 is: “Turn a detected parcel problem into an actionable and trackable resolution process.” It has three parts. *Actionable* points to requirements about workflow, responsibility and actions; *trackable* points to status, monitoring and escalation; and *resolution process* points to resolution, verification and closure. The lifecycle Detect → Understand → Act → Monitor → Verify Resolution → Close is the structure that A2 uses to organise these goals, and

A4 reused it as the basis of the behavioural model. Table 2 gives the category-level mapping to the A3 requirements.

A.1.3 Functional ↔ Non-Functional Requirements

Non-functional requirements constrain how functional ones must behave. The relationship that is explicitly supported by A4 is NFR-15 (integration-failure handling): the A4 review showed that functions which depend on external systems need a defined behaviour when the integration is unavailable. NFR-15 therefore *constrains* the functional requirements that depend on integrations (M-14).^[confirm] Further FR/NFR pairs, for example NFRs about auditability or response time and the FRs for evidence, status and monitoring, are candidates that we have not yet checked one by one. We keep them out of the matrix rather than assert them.

A.1.4 Requirements → Assignment 4 Review Findings

A4 mapped requirements to elements of the behavioural model, and used VQ-01 to VQ-14 and S-01 to S-12 to review them. The review produced decisions C-01 to C-10. The findings that concern requirements directly were: the waiting-for-customer-information state (FR-22), integration-failure handling (NFR-15), a workflow fallback, repeated escalation, and closure verification. The chain is: requirement → model element → scenario or validation question → finding → review decision. It makes it possible to say which requirements were challenged during validation and what was decided (M-08 to M-12).

A.1.5 Original → Revised Requirements

The A4 findings imply changes to the A3 requirements. Table 1 lists the findings and the requirement each one concerns. Where the A4 text does not name the requirement ID, we mark it as to be confirmed. Keeping the original and the revised version linked (*evolves-to*) shows why a requirement changed and prevents the A3 wording from being mistaken for the current one.

Table 1: A4 findings and the requirements they concern.

A4 finding	Requirement	Direction of revision
Waiting-for-customer-information state	FR-22	The incident lifecycle needs an explicit state for the case where progress depends on information from the customer.
Integration-failure handling	NFR-15	Behaviour when an integration fails must be defined.
Workflow fallback	FR-09 (cand.) ^[confirm]	A fallback is needed when no workflow can be selected.
Repeated escalation	Escalation FR ^[confirm]	Behaviour for escalating the same incident more than once must be defined.
Closure verification	Closure FR ^[confirm]	Closure must depend on verified resolution.

A.1.6 Requirements → Lifecycle Stages / Scenarios

Table 2 connects the requirement categories of A3 to the lifecycle stages. This is a category-level mapping that we derived from the stage names and the category names; the ID-level assignment

is done in the matrix where needed. Scenarios S-01 to S-12 exercise paths through these stages and so give a second route from the lifecycle to the requirements.

Table 2: Requirement categories (A3) mapped to lifecycle stages (A2). Candidate mapping at category level.

Lifecycle stage	A3 requirement categories
Detect	Incident initiation
Understand	Classification; information and status
Act	Workflow; action and responsibility; evidence
Monitor	Monitoring; escalation; customer communication
Verify Resolution, Close	Resolution and closure
Across stages	Operational insights; non-functional requirements

A.1.7 Functional Requirement Dependencies

Some functional requirements only make sense in combination with others. We identified the following dependencies:

- **FR-04 → FR-09.** The classification of an incident (FR-04) is an input to the workflow selection (FR-09). This is the dependency used in the impact-analysis example in Section C.2.
- **FR-11, FR-12 → FR-13.** FR-13 relies on what FR-11 and FR-12 provide. [confirm]
- **FR-19 → FR-20.** FR-20 builds on FR-19. [confirm]

The rationale of the last two should be written with the actual requirement titles from A3 before submission. Dependencies matter because a change to the source requirement may also change the dependent one.

B Traceability Representation

B.1 Traceability Matrix

Table 3 is a representative matrix. It does not contain all possible links; it contains links that demonstrate each relationship type from Section A. Each row states why the relationship exists and where the evidence is.

Table 3: Representative traceability matrix.

ID	Source artefact	Source ID	Link type	Target artefact	Target ID	Evidence / rationale
M-01	Stakeholder	ST-005	motivates	NFR	NFR-15	[CAND.] IT / Integration Team is affected by integration behaviour; A4 raised integration-failure handling.
M-02	Product goal	A2 value statement	satisfied-by	Lifecycle stages	Detect... Close	[A2] The lifecycle structures the delivery of the “actionable and trackable” value.

continued on next page

ID	Source fact	arte-	Source ID	Link type	Target fact	arte-	Target ID	Evidence / rationale
M-03	Lifecycle stage		Understand	covers	FR		FR-04	[CAND.] Classification belongs to understanding the problem (category-level mapping).
M-04	FR		FR-04	depends-on (input to)	FR		FR-09	[A3] Workflow selection uses the classification result.
M-05	FR		FR-11, FR-12	depends-on (input to)	FR		FR-13	[A3] Dependency identified in requirements analysis. [confirm]
M-06	FR		FR-19	depends-on (input to)	FR		FR-20	[A3] Dependency identified in requirements analysis. [confirm]
M-07	Scenario		S-03	validates	FR		FR-09	[A4] S-03 exercises the unresolved-workflow path of workflow selection.
M-08	Model ment	ele-	Waiting for customer information	realises	FR		FR-22	[A4] Model state reviewed in A4 for FR-22.
M-09	FR		FR-22	concerns (finding)	Review finding / decision	C-??	[con- firm]	[A4] Waiting-for-customer-information state.
M-10	NFR		NFR-15	concerns (finding)	Review finding / decision	C-??	[con- firm]	[A4] Integration-failure handling.
M-11	FR		FR-09	concerns (finding)	Review finding / decision	C-??	[con- firm]	[CAND.] Workflow fallback finding; requirement ID to be confirmed.
M-12	FR		Escalation / closure FRs	concerns (finding)	Review finding / decision	C-??	[con- firm]	[A4] Repeated escalation and closure verification findings; IDs to be added.
M-13	Review finding	find-	C-?? (FR-22) [con- firm]	refined-by	Revised requirement		FR-22 (rev.)	[CAND.] Revised wording to be recorded as a new version.
M-14	NFR		NFR-15	constrains	FR		FR-?? (integration-dependent) [confirm]	[CAND.] Functions depending on integrations need defined failure behaviour.

B.2 Traceability Information Model

The traceability information model (TIM) defines which artefact types and link types we use. Table 4 lists the artefact types, and Table 5 lists the link types. Link directions are written from the upstream to the downstream artefact, and the inverse name is given in brackets.

B.3 Traceability Chain

Figure 1 shows the overall chain that the TIM describes. Arrows are labelled in the downward direction, and the dashed arrow shows that the revised requirement becomes the current version and enters the chain again.

Table 4: Artefact types in the traceability information model.

Artefact type	Origin	Examples
Stakeholder / need	A2	ST-001 to ST-005
Product goal / vision	A2	Value statement; lifecycle Detect → Close
Functional requirement	A3	FR-01 to FR-28
Non-functional requirement	A3	NFR-01 to NFR-16
Behavioural model element	A4	Lifecycle states, e.g. waiting for customer information
Validation question	A4	VQ-01 to VQ-14
Scenario	A4	S-01 to S-12
Review finding / decision	A4	C-01 to C-10
Revised requirement	A4 / A5	New version of an FR or NFR after review
Software / prototype component	Future	Not yet defined; reserved for later assignments

Table 5: Link types in the traceability information model.

Link type	From → To	Purpose
motivates (motivated-by)	Stakeholder need → Product goal	Shows who a goal is for.
satisfied-by (satisfies)	Product goal → Requirement	Justifies a requirement by a goal.
covers (covered-by)	Lifecycle stage → Requirement	Locates a requirement in the lifecycle.
realised-in (realises)	Requirement → Model element	Shows how a requirement appears in the behavioural model.
validated-by (validates)	Requirement / model element → Scenario, VQ	Shows how a requirement was checked.
raises (raised-by)	Scenario / VQ → Review finding	Records where a problem was found.
concerns (concerned-by)	Review finding → Requirement	Identifies the requirement a finding is about.
refined-by (refines)	Review finding → Revised requirement	Explains why a requirement changed.
evolves-to (evolves-from)	Requirement → Revised requirement	Keeps the version history of a requirement.
depends-on (required-by)	FR → FR	Supports impact analysis between requirements.
constrains (constrained-by)	NFR → FR	Shows which function a quality requirement limits.
complements	FR ↔ FR / NFR	Marks requirements that are meant to be read together.
implemented-by (implements)	Requirement → Component	Reserved for later assignments; not used yet.

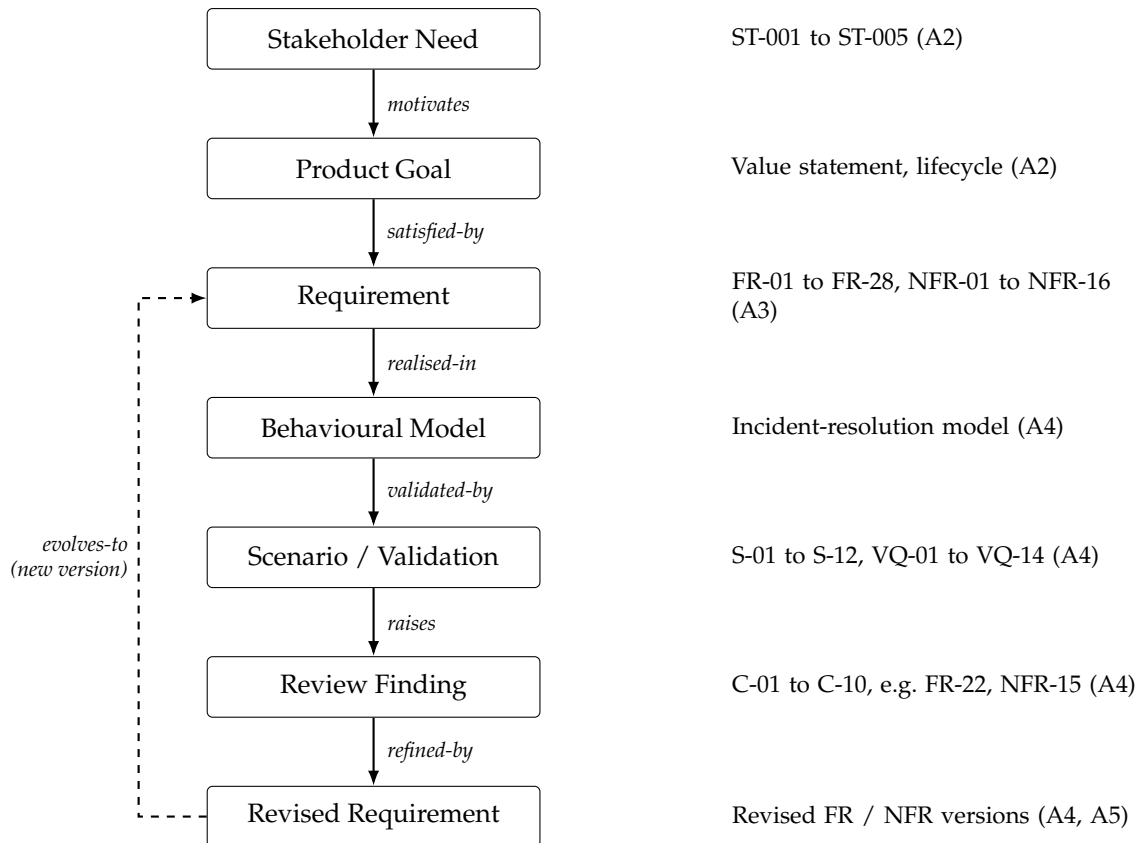


Figure 1: Overall traceability chain for ParcelResolve, with the artefacts of each step.

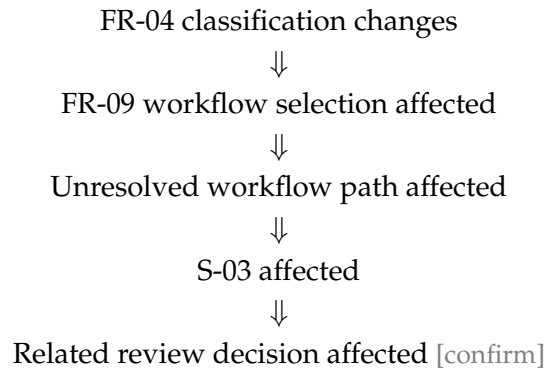
C Why the Selected Trace Links Are Useful

C.1 Requirements Understanding

A requirement such as FR-09 cannot be understood in isolation. Following its links shows that it takes the result of FR-04 as input (M-04), that scenario S-03 checks its unresolved-workflow path (M-07), and that a finding about a workflow fallback was raised (M-11). A new team member reading FR-09 can see where it comes from and how it was reviewed, which the requirement text alone does not tell.

C.2 Change Management and Impact Analysis

When a requirement changes, the links show what else must be checked. The example below follows FR-04 forward:



If, for instance, the set of classification values in FR-04 is extended, FR-09 must define which workflow applies to the new value, the unresolved-workflow path of FR-09 may change, S-03 must be re-run or rewritten, and the review decision about the workflow fallback may no longer be valid.

Impact analysis works in both directions. *Forward* analysis starts from a changed artefact and finds what depends on it, as above. *Backward* analysis starts from an affected artefact and finds the causes. If S-03 fails after a change, the backward links lead to FR-09 and then to FR-04, so the group can see which requirement change is responsible.

C.3 Requirements Review and Validation

Links make gaps visible during review. In A4 each finding was attached to scenarios and model elements; with the links in Table 3 the reverse question can also be asked: which requirements were *never* touched by a scenario or validation question? A requirement with no validates link has not been checked by the A4 model and is a candidate for the next review. Likewise, FR-22 is linked to a model state and to a finding, so a reviewer can follow the whole validation history of the requirement.

C.4 Maintenance and Evolution

Maintenance means keeping the specification correct after it has been released. NFR-15 shows the problem: it constrains every function that uses an integration. If the integration landscape changes, the *constrains* links give the list of functional requirements that must be reconsidered (M-14), instead of searching the full document. The evolves-to links also keep the old and the new version of a requirement together, so the reason for a change is not lost.

C.5 Scope Control

Every requirement should trace upward to a stakeholder need or product goal. A requirement or a new request without such a link is a scope-creep candidate. For example, a hypothetical request for a feature outside the lifecycle Detect → Close would not connect to the A2 value statement (M-02) and would have to be justified before being added. Conversely, a stakeholder need without any requirement points to a gap in scope.

D Tool / Approach Supporting Requirements Traceability

D.1 Existing Requirements Management Tools

Enterprise requirements management tools such as Jama Connect, IBM DOORS Next, Siemens Polarion and PTC Codebeamer support traceability through typed links between artefacts, trace views and matrices, and, in general, mechanisms that flag links as suspect when a linked item changes. These tools are designed for large, regulated projects and require licences and set-up. We did not evaluate them hands-on, and for a small student project they are more than we need, so we investigate a lightweight approach instead.

D.2 LLM-Supported Traceability Approach

Our approach combines a plain-text traceability matrix with LLM suggestions that a human reviews. It has seven elements:

1. **Stable identifiers.** Every artefact keeps its ID (ST-, FR-, NFR-, VQ-, S-, C-). IDs are never reused, and revised requirements keep the ID with a version number.
2. **Central traceability matrix.** All links are kept in one table (Table 3 is an extract), with the columns of Section B.1.
3. **Versioned traceability information.** The matrix is stored in a text format under version control, so changes to links can be compared like changes to requirements.
4. **LLM-assisted candidate-link detection.** The LLM receives two artefacts, for example a changed requirement and a list of scenarios, and proposes possible links with a short justification. It only proposes; it does not write to the matrix.
5. **Human validation.** A group member checks every proposed link against the source text and accepts, changes or rejects it.
6. **Suspect-link detection.** When a requirement changes, all links that start or end at it are marked suspect.
7. **Revision records.** Each decision on a link is recorded with the reason and date, in the same way as in the decision record in Section G.

The maintenance procedure after a change is shown in Table 6.

Table 6: Maintenance of links after a requirement change.

Step	Action	Description
1	Requirement changes	A requirement is edited and receives a new version.
2	Related links become suspect	All links of the requirement are marked suspect in the matrix.
3	Identify affected artefacts	Follow the links to find dependent requirements, scenarios and decisions; the LLM may suggest further candidates.
4	Re-evaluate links	A human reads source and target again and checks whether the relationship still holds.
5	Keep / modify / remove	Each link is kept, changed or removed.
6	Record decision	Reason and date are written to the revision record.
7	Update matrix	The matrix is updated and the suspect marks are cleared.

Example. Suppose the classification in FR-04 is reworded. The links M-04 (FR-04 → FR-09) and everything attached to FR-04 become suspect. The LLM is given the new FR-04 together with FR-09 and S-03 and asked whether the link still holds; it may also suggest that another requirement now depends on FR-04. The group reads FR-04 and FR-09, keeps M-04, modifies the evidence text of M-07 because the unresolved path of FR-09 changed, and rejects a proposed link that the LLM could not justify. The three decisions are written to the revision record and the matrix is updated.

A record in the matrix needs these fields: link ID, source, target, link type, status (valid / suspect / candidate / removed), rationale, evidence, requirement version, reviewer and date.

D.3 Benefits and Challenges

Benefits.

- Missing links are found faster, because the LLM proposes candidates that a reader might overlook.
- Impact analysis is easier, since affected artefacts are found by following links.
- The decision history is preserved in the revision records.
- The approach scales to a larger number of artefacts than manual reading.

Challenges.

- Maintaining the matrix by hand costs effort, and it is easy to skip when time is short.
- Link drift: if links are not updated, they silently become wrong.
- An LLM can propose links that are hallucinated or unjustified.
- Ambiguous requirement wording produces ambiguous candidate links.
- Human validation remains necessary, so the effort is reduced but not removed.
- Same-LLM self-review: if the same model drafts an artefact and then reviews it, errors may go unnoticed. We therefore use a human reviewer, and where possible a different tool for review.

E Maintaining Trace Links as Requirements Evolve — Literature Review

We reviewed six sources. Their bibliographic data was checked against publisher, DBLP or arXiv pages; for the sources marked as preprints the version used is the arXiv version. Our summaries are based on the abstracts and the available chapter or paper descriptions, not on a full reading of every paper. Table 7 summarises them.

Table 7: Literature reviewed.

Source	Year	Research focus	Relevant finding	Relevance to ParcelResolve
Gotel et al. [1]	2012	Foundations of software and systems traceability	Defines traceability and its purposes, including following artefacts forward and backward through the life cycle.	Basis for forward and backward impact analysis (Section C.2).

Source	Year	Research focus	Relevant finding	Relevance to ParcelResolve
Mäder & Gotel [2]	2012	Maintaining links when models change	Up-to-date relations are needed, and decay must be prevented; change events are matched with rules to update impacted links semi-automatically.	Supports our suspect-link procedure (Table 6); our version is manual and simpler.
Rodriguez et al. [4]	2023	Challenges of deploying live (near real-time) automated traceability	Traceability is labour-intensive and error-prone; keeping automated traceability current in practice raises challenges.	Confirms that maintenance cost is the main problem and that automation is not trivial.
Lin et al. [3]	2021	Transformer (BERT) models for link generation	Pre-trained models give much higher accuracy for links between natural-language artefacts and code than earlier methods.	Shows the potential of ML, but results are for code-text links with training data; ParcelResolve has neither.
Rodriguez et al. [5]	2023	Prompt engineering for LLM-based traceability	LLMs can predict links; prompt refinement improves ranking over zero-shot prompts, but best use is unclear.	Guides how we formulate the LLM request for candidate links (D.2, element 4).
Alturayef et al. [6]	2026	TraceLLM: prompts and demonstrations for requirements traceability	Performance depends strongly on prompt quality; results support semi-automated workflows where analysts review the candidate links.	Directly supports “LLM suggests, human decides”.

E.1 Traceability and Maintenance / Change Management

Gotel et al. [1] describe traceability as the ability to follow an artefact in both directions, which is what we use for forward and backward impact analysis. Mäder and Gotel [2] point out that traceability only brings its benefits when the relations are current, and that they decay as the system evolves unless they are maintained. They propose to detect change events and update affected relations by rules. Their approach works on UML models, but the idea of reacting to a change event is the same as our suspect-link step.

E.2 Automated / ML / LLM-Based Traceability

Lin et al. [3] show that pre-trained transformer models can generate links much more accurately than earlier approaches. Rodriguez et al. [5] and Alturayef et al. [6] study LLMs directly. Both find that the quality of the prompt strongly affects the result, and Rodriguez et al. state that the best way to use LLMs for traceability is still unclear. Alturayef et al. conclude that their results support workflows in which analysts review candidate links.

E.3 Continuous or Evolving Traceability

Rodriguez et al. [4] discuss what is needed to keep automated traceability up to date while a project changes. Together with [2] this shows that creating links once is not the main problem. Keeping them valid over time is.

E.4 Implications for ParcelResolve

The literature supports the following argument. Traceability supports change management [1]. Maintaining links is costly [2, 4], so automation can reduce the effort [3]. However, automated and LLM-based link generation is not fully reliable and depends on prompts [5, 6], so human validation is necessary. ParcelResolve therefore uses LLM suggestions combined with human review (Section D.2). We do not claim that the LLM maintains traceability; it proposes links and a group member decides.

Two limits of the literature for our case should be stated. Most of the work concerns links between requirements and code, and ParcelResolve has no implementation yet, so only requirement-level and model-level links are relevant now. Also, the studied models were trained or tested on larger artefact sets than our 44 requirements.

F Observations and Conclusions

Relationships established. We identified relationships from stakeholders and product goals to requirements, between functional and non-functional requirements, from requirements to the A4 model, scenarios and findings, and from original to revised requirements. Several links are documented in A2–A4; others are marked as candidates because the earlier documents do not yet state them.

Matrix and TIM. The matrix (Table 3) and the TIM (Tables 4 and 5) give us a common vocabulary for artefacts and links. They allowed us to do an impact analysis for a change of FR-04 (Section C.2) and to see which requirements were affected by A4 findings.

Maintenance. Links are kept current by marking them suspect after a change, re-evaluating them, and recording the decision (Table 6). The LLM supports this by proposing candidates.

Benefits and limitations. The approach finds gaps and shows the effect of changes, but it costs manual effort, and LLM suggestions can be wrong. The matrix in this report is representative, not complete.

Open for future assignments.

- Resolve all candidate links and the [confirm]marks against the A2–A4 documents.
- Extend the matrix to the remaining FR and NFR, and add the missing FR/NFR pairs.
- Add implementation links once components or a prototype exist.
- Try the candidate-link procedure on a real change and report how many LLM proposals were accepted.

G Review Decision Record

This record is separate from the analysis above. SELF means a decision or contribution of the group. AI PROPOSAL means a suggestion from the LLM. The decision column uses ACCEPTED, DEFERRED or NOTED. AI suggestions are not merged into our own analysis unless they are accepted here.

Table 8: Review decision record.

ID	Source	Proposal	Decision	Reason
R-01	AI PRO- POSAL	Use a representative matrix instead of an exhaustive one.	ACCEPTED	Matches the assignment goal of a small matrix.
R-02	AI PRO- POSAL	Tag each link as documented (A2–A4) or candidate.	ACCEPTED	Avoids claiming links that earlier work does not support.
R-03	SELF	Use FR-04 → FR-09 → S-03 as the impact-analysis example.	ACCEPTED	Based on our own analysis of the requirements and scenarios.
R-04	AI PRO- POSAL	Link ST-005 to NFR-15 as a motivating stakeholder.	DEFERRED	Needs the exact A2 need statement.
R-05	AI PRO- POSAL	Treat FR-09 as the requirement concerned by the workflow-fallback finding.	DEFERRED	Requirement ID not confirmed in A4.
R-06	AI PRO- POSAL	Map requirement categories to lifecycle stages at category level (Table 2).	DEFERRED	Must be checked against the A4 model mapping.
R-07	AI PRO- POSAL	Rationale for dependencies FR-11/FR-12 → FR-13 and FR-19 → FR-20.	DEFERRED	Rationale to be written from the A3 text.
R-08	AI PRO- POSAL	Select six sources for the literature review.	ACCEPTED	Bibliographic data checked; summaries based on abstracts only, full reading still open.
R-09	NOTED	LLM-proposed links can be unjustified; every link needs human review.	NOTED	Part of the approach in Section D.2.
R-10	NOTED	The same LLM family drafted and reviewed parts of this report.	NOTED	Limits independent review; human reading of the final text is required.

References

- [1] O. Gotel, J. Cleland-Huang, J. Huffman Hayes, A. Zisman, A. Egyed, P. Grünbacher, A. Dekhtyar, G. Antoniol, J. Maletic, and P. Mäder. Traceability fundamentals. In *Software and Systems Traceability*, pp. 3–22. Springer, 2012. doi:10.1007/978-1-4471-2239-5_1.
- [2] P. Mäder and O. Gotel. Towards automated traceability maintenance. *Journal of Systems and Software*, 85(10):2205–2227, 2012. doi:10.1016/j.jss.2011.10.023.
- [3] J. Lin, Y. Liu, Q. Zeng, M. Jiang, and J. Cleland-Huang. Traceability transformed: Generating more accurate links with pre-trained BERT models. In *Proc. 43rd International Conference on Software Engineering (ICSE)*, 2021. arXiv:2102.04411.
- [4] A. D. Rodriguez, K. R. Dearstyne, and J. Cleland-Huang. Understanding the challenges of deploying live-traceability solutions. arXiv preprint arXiv:2306.10972, 2023.
- [5] A. D. Rodriguez, K. R. Dearstyne, and J. Cleland-Huang. Prompts matter: Insights and strategies for prompt engineering in automated software traceability. In *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*, 2023. arXiv:2308.00229.

- [6] N. Alturayef, I. Ahmad, and J. Hassine. TraceLLM: Leveraging large language models with prompt engineering for enhanced requirements traceability. arXiv preprint arXiv:2602.01253, 2026.

AI Assistance Disclosure

An LLM (Claude) was used to draft the structure and text of this report and the LaTeX source, to propose candidate trace links, and to check bibliographic details of the cited sources. The group is responsible for the content. The AI proposals and our decisions on them are listed separately in Section G. All links marked as candidates, and all literature summaries, must be checked against the original documents and papers by the group.